

UIDE LÓGICA DE PROGRAMACIÓN

NOMBRE:

Tripuliche Ronquillo Lizbeth
Alvarado Ruiz Janneli

DESCRIPCIÓN DE LA ASIGNACIÓN:

Desafío:

Desarrollar una mejora analítica que permita cargar los registros desde un archivo, verificar la calidad de los datos, corregir o excluir registros inválidos, calcular indicadores útiles y presentar resultados mediante gráficos.

ENLACE DE GOOGLE COLAB:

<https://colab.research.google.com/drive/1LweXNUQOOizhjoxoWMgPDdztc0UPMCgn?usp=sharing>

EXPLICACIÓN BREVE DE LO REALIZADO:

En esta actividad se desarrolló una mejora analítica para el registro de mantenimientos realizados a distintos equipos de una institución. Para ello, se implementó un programa en Python que permite leer archivos CSV y procesar la información utilizando las bibliotecas pandas, numpy y matplotlib.

Durante el desarrollo del programa, se incorporaron diferentes mecanismos de validación para detectar errores en los datos, como códigos vacíos o duplicados, fechas incorrectas, costos negativos, duraciones no válidas, valores de satisfacción fuera del rango establecido, estados no permitidos y categorías de equipos no contempladas. Los registros que presentaban errores fueron identificados y eliminados para generar un conjunto de datos limpio.

Además, se utilizaron funciones de NumPy para realizar cálculos estadísticos, como la media, la mediana y el percentil de los costos. Posteriormente, mediante pandas, se calcularon diferentes indicadores, entre ellos el número de mantenimientos válidos, el costo total, el costo promedio, la cantidad de mantenimientos por área y el costo promedio según el tipo de mantenimiento.

También se implementó un ciclo for para recorrer las reglas de validación y una estructura if, elif y else para analizar la cantidad de errores encontrados en cada caso. Finalmente, se generaron gráficos para facilitar la interpretación de los resultados y se exportaron los archivos generados por el programa, cumpliendo con todos los requisitos establecidos en la actividad.

ENTRADAS:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. LECTURA Y MANEJO DE ARCHIVOS

def leer_archivo(ruta):
    try:
        datos = pd.read_csv(ruta)

        if datos.empty:
            print(f"\nERROR: El archivo '{ruta}' está vacío.")
            return None
        else:
            print(f"\nArchivo leído correctamente: {ruta}")
            return datos
    except FileNotFoundError:
        print(f"\nERROR: El archivo '{ruta}' no existe.")
        return None
    except pd.errors.EmptyDataError:
        print(f"\nERROR: El archivo '{ruta}' está vacío.")
```

```
except pd.errors.ParserError:
    print(f"\nERROR: El archivo '{ruta}' no tiene un formato CSV válido.")
    return None

except Exception as e:
    print(f"\nERROR inesperado al leer '{ruta}': {e}")
    return None

# 2. VALIDACIÓN Y LIMPIEZA

def validar_y_limpiar(df):
    df = df.copy()

    equipos_validos = [
        "Switch",
        "Router",
        "Proyector",
        "Desktop",
        "Laptop",
        "Impresora"
    ]

    estados_validos = [
        "Completado",
        "Pendiente",
        "En proceso"
    ]
```

```
df["fecha_temp"] = pd.to_datetime(
    df["fecha"],
    errors="coerce"
)

df["costo_temp"] = pd.to_numeric(
    df["costo"],
    errors="coerce"
)

df["duracion_temp"] = pd.to_numeric(
    df["duracion_horas"],
    errors="coerce"
)

df["satisfaccion_temp"] = pd.to_numeric(
    df["satisfaccion"],
    errors="coerce"
)

validaciones = {

    "Códigos vacíos":
        df["codigo"].isna()
        |
        (df["codigo"].astype(str).str.strip() == ""),

    "Códigos duplicados":
        df["codigo"].duplicated(),

    "Fechas inválidas":
        df["fecha_temp"].isna(),
```

```
"Costos inválidos":
    df["costo_temp"].isna()
    |
    (df["costo_temp"] < 0),

    "Duraciones inválidas":
        df["duracion_temp"].isna()
        |
        (df["duracion_temp"] <= 0),

    "Satisfacción fuera del rango":
        ~df["satisfaccion_temp"].between(1, 5),

    "Estados inválidos":
        ~df["estado"].isin(estados_validos),

    "Equipos no contemplados":
        ~df["equipo"].isin(equipos_validos),

    "Áreas vacías":
        df["area"].isna()
        |
        (df["area"].astype(str).str.strip() == "")
}

filas_invalidas = pd.Series(
    False,
    index=df.index
)

errores = []

print("\nERRORES DETECTADOS:")
```

```

# CICLO FOR Y ESTRUCTURA IF / ELIF / ELSE
for nombre, condicion in validaciones.items():
    cantidad = condicion.sum()

    if cantidad == 0:
        print(f"- {nombre}: 0 errores")

    elif cantidad == 1:
        print(f"- {nombre}: 1 error")
        errores.append(nombre)
        filas_invalidas = filas_invalidas | condicion

    else:
        print(f"- {nombre}: {cantidad} errores")
        errores.append(nombre)
        filas_invalidas = filas_invalidas | condicion

if len(errores) == 0:
    print("\nNo se encontraron errores.")
else:
    print(
        f"\nTotal de tipos de errores detectados: {len(errores)}"
    )

```

```

df_limpio = df[~filas_invalidas].copy()

df_limpio["fecha"] = df_limpio["fecha_temp"]
df_limpio["costo"] = df_limpio["costo_temp"]
df_limpio["duracion_horas"] = df_limpio["duracion_temp"]
df_limpio["satisfaccion"] = df_limpio["satisfaccion_temp"]

df_limpio.drop(
    columns=[
        "fecha_temp",
        "costo_temp",
        "duracion_temp",
        "satisfaccion_temp"
    ],
    inplace=True
)

print(
    f"\nCantidad de registros iniciales: {len(df)}"
)

print(
    f"Cantidad de registros válidos tras la limpieza: {len(df_limpio)}"
)

return df_limpio

# 3. ANÁLISIS CON NUMPY, PANDAS, GRÁFICOS Y EXPORTACIÓN
def analizar_y_exportar(df_limpio):

```

```

print("\n--- OPERACIONES CON NUMPY ---")

media_costo = np.mean(
    df_limpio["costo"]
)

mediana_costo = np.median(
    df_limpio["costo"]
)

percentil_75 = np.percentile(
    df_limpio["costo"],
    75
)

df_limpio["categoria_costo"] = np.where(
    df_limpio["costo"] >= media_costo,
    "Alto",
    "Bajo"
)

print(
    f"Media del costo: ${media_costo:.2f}"
)

print(
    f"Mediana del costo: ${mediana_costo:.2f}"
)

print(
    f"Percentil 75 del costo: ${percentil_75:.2f}"
)

```

```

print("\n--- INDICADORES CON PANDAS ---")

print(
    f"Número de mantenimientos válidos: {len(df_limpio)}"
)

print(
    f"Costo total: ${df_limpio['costo'].sum():.2f}"
)

print(
    f"Costo promedio: ${df_limpio['costo'].mean():.2f}"
)

mantenimiento_costoso = df_limpio.loc[
    df_limpio["costo"].idxmax()
]

print("\nMantenimiento más costoso:")

print(
    f"Código: {mantenimiento_costoso['codigo']}"
)

print(
    f"Equipo: {mantenimiento_costoso['equipo']}"
)

print(
    f"Costo: ${mantenimiento_costoso['costo']:.2f}"
)

print("\nCantidad de mantenimientos por área:")

```

```

print(
    df_limpio["area"].value_counts()
)

print(
    "\nCosto promedio por tipo de mantenimiento:"
)

print(
    df_limpio
    .groupby("tipo_mantenimiento")["costo"]
    .mean()
    .map("${:.2f}".format)
)

# GRÁFICO 1

costo_area = (
    df_limpio
    .groupby("area")["costo"]
    .sum()
)

plt.figure(figsize=(8, 4))

costo_area.plot(
    kind="bar",
    color="skyblue",
    edgecolor="black"
)

plt.title(
    "Costo total de mantenimientos por área"
)

```

```

plt.xlabel("Área")

plt.ylabel("Costo ($)")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# GRÁFICO 2

cantidad_tipo = (
    df_limpio["tipo_mantenimiento"]
    .value_counts()
)

plt.figure(figsize=(8, 4))

cantidad_tipo.plot(
    kind="bar",
    color="orange",
    edgecolor="black"
)

plt.title(
    "Cantidad de mantenimientos por tipo"
)

plt.xlabel(
    "Tipo de mantenimiento"
)

plt.ylabel(
    "Cantidad"
)

```

```

plt.xticks(rotation=0)

plt.tight_layout()

plt.show()

# EXPORTACIÓN

df_limpio.to_csv(
    "mantenimientos_limpios.csv",
    index=False
)

resumen_area = (
    df_limpio
    .groupby("area")
    .agg(
        cantidad=(
            "codigo",
            "count"
        ),
        costo_total=(
            "costo",
            "sum"
        ),
        costo_promedio=(
            "costo",
            "mean"
        )
    )
)

resumen_area.to_csv(
    "resumen_por_area.csv"
)

```

```

print(
    "\nArchivos exportados correctamente."
)

# 4. PRUEBAS OBLIGATORIAS

print("=" * 60)

print(
    "PRUEBA 1: ARCHIVO VÁLIDO (mantenimientos.csv)"
)

print("=" * 60)

df_valido = leer_archivo(
    "mantenimientos.csv"
)

if df_valido is not None:

    df_limpio = validar_y_limpiar(
        df_valido
    )

    analizar_y_exportar(
        df_limpio
    )

print("\n" + "=" * 60)

print(
    "PRUEBA 2: ARCHIVO CON ERRORES (mantenimientos_con_errores.csv)"
)

```

```

)

print("=" * 60)

df_error = leer_archivo(
    "mantenimientos_con_errores.csv"
)

if df_error is not None:

    validar_y_limpiar(
        df_error
    )

print("\n" + "=" * 60)

print(
    "PRUEBA 3: ARCHIVO INEXISTENTE (archivo_inexistente.csv)"
)

print("=" * 60)

leer_archivo(
    "archivo_inexistente.csv"
)

```

SALIDAS:

```
*** =====
PRUEBA 1: ARCHIVO VÁLIDO (mantenimientos.csv)
=====

Archivo leído correctamente: mantenimientos.csv

ERRORES DETECTADOS:
- Códigos vacíos: 0 errores
- Códigos duplicados: 0 errores
- Fechas inválidas: 0 errores
- Costos inválidos: 0 errores
- Duraciones inválidas: 0 errores
- Satisfacción fuera del rango: 0 errores
- Estados inválidos: 0 errores
- Equipos no contemplados: 0 errores
- Áreas vacías: 0 errores

No se encontraron errores.

Cantidad de registros iniciales: 120
Cantidad de registros válidos tras la limpieza: 120

--- OPERACIONES CON NUMPY ---
Media del costo: $84.84
Mediana del costo: $83.96
Percentil 75 del costo: $113.60

--- INDICADORES CON PANDAS ---
Número de mantenimientos válidos: 120
Costo total: $10,180.53
Costo promedio: $84.84
```

Mantenimiento más costoso:

Código: MNT-0119
Equipo: Proyector
Costo: \$172.63

Cantidad de mantenimientos por área:

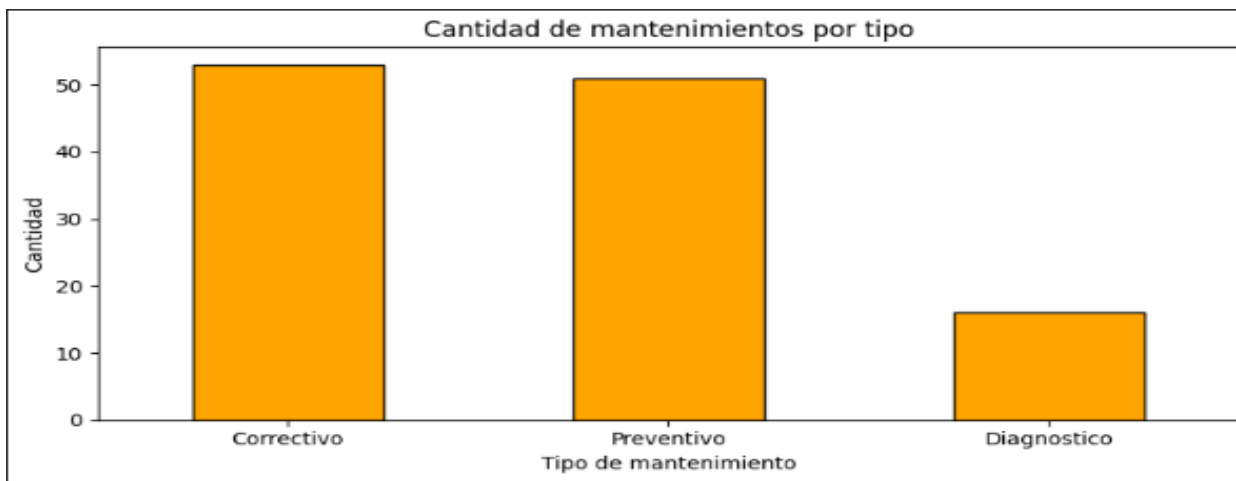
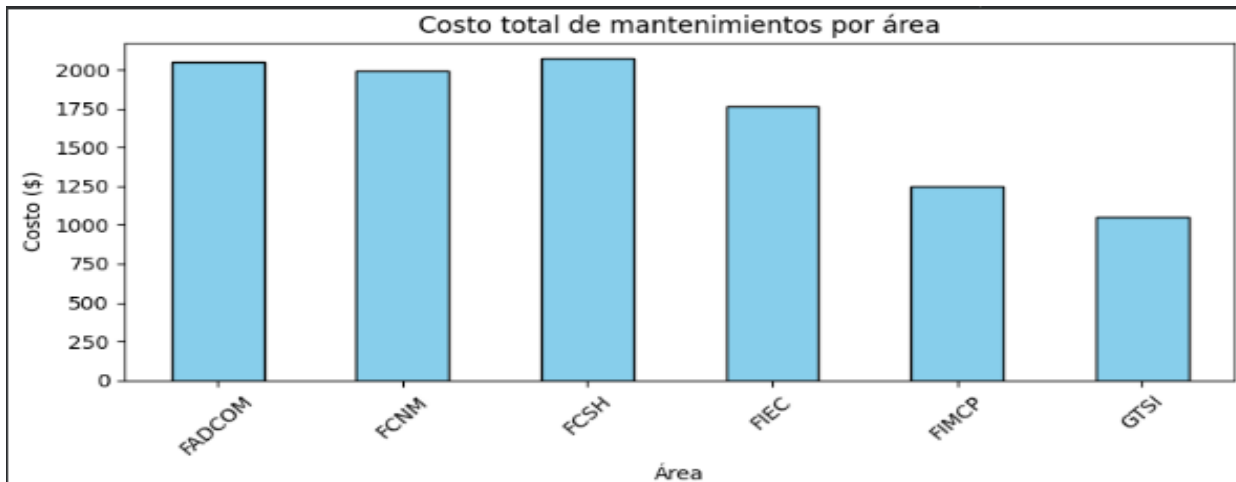
area	count
FCSH	28
FADCOM	22
FIEC	22
FCNM	22
GTSI	13
FIMCP	13

Name: count, dtype: int64

Costo promedio por tipo de mantenimiento:

tipo_mantenimiento	costo
Correctivo	\$117.82
Diagnostico	\$49.83
Preventivo	\$61.54

Name: costo, dtype: object



Archivos exportados correctamente.

=====

PRUEBA 2: ARCHIVO CON ERRORES (mantenimientos_con_errores.csv)

=====

Archivo leído correctamente: mantenimientos_con_errores.csv

ERRORES DETECTADOS:

- Códigos vacíos: 1 error
- Códigos duplicados: 1 error
- Fechas inválidas: 1 error
- Costos inválidos: 2 errores
- Duraciones inválidas: 1 error
- Satisfacción fuera del rango: 1 error
- Estados inválidos: 1 error
- Equipos no contemplados: 1 error
- Áreas vacías: 1 error

Total de tipos de errores detectados: 9

Cantidad de registros iniciales: 120

Cantidad de registros válidos tras la limpieza: 110

=====

PRUEBA 3: ARCHIVO INEXISTENTE (archivo_inexistente.csv)

=====

ERROR: El archivo 'archivo_inexistente.csv' no existe.

README

1. Lectura y manejo de archivos
2. Validación y limpieza de los datos
3. Implementación de ciclos y estructuras de control
4. Uso de NumPy
5. Análisis con pandas
6. Visualización de datos
7. Exportación de resultados
8. Pruebas realizadas